

Manual de Estudio Profundo Metaheurísticas para el LRP

Algoritmos: Fuerza Bruta, ALNS y MA|PM

Con pseudocódigos, ejemplos paso a paso
y guía de defensa ante el jurado

Sebastián Lizárraga Calderón
Johannes Loayza Huamán

Índice

1. El Problema LRP en pocas palabras	3
2. Algoritmo de Fuerza Bruta (Solución Exacta)	3
2.1. ¿Qué es y para qué sirve?	3
2.2. Estructura del algoritmo	4
2.3. Held-Karp: cómo funciona	4
2.4. Set-Cover DP: cómo funciona	5
2.5. Pseudocódigo completo de la Fuerza Bruta	6
3. ALNS: Adaptive Large Neighborhood Search	11
3.1. Idea central y motivación	11
3.2. Representación de la solución	11
3.3. Operadores de Destrucción	12
3.3.1. 1. Random Removal (Destrucción Aleatoria)	12
3.3.2. 2. Worst Removal (Destrucción de los Peores)	12
3.3.3. 3. Shaw Removal (Destrucción por Similitud Geográfica)	13
3.3.4. 4. Route Removal (Destrucción de Ruta Completa)	13
3.3.5. 5. Depot Closing (Cierre de Depósito)	14
3.4. Operadores de Reparación	14
3.4.1. 1. Greedy Insertion (Inserción Codiciosa)	14
3.4.2. 2. Regret-2 Insertion (Inserción con Arrepentimiento)	15
3.5. Ruleta Adaptativa (el mecanismo de aprendizaje)	16
3.6. Simulated Annealing (criterio de aceptación)	17
3.7. Pseudocódigo ALNS con explicación línea por línea	18
4. MA PM: Algoritmo Memético con Gestión de Población	19
4.1. ¿Qué significan MA y PM?	19
4.2. El Cromosoma: Giant Tour + Estado de Depósitos	20
4.3. El Algoritmo Split: corazón del MA PM	20
4.3.1. Intuición	20
4.3.2. Construcción del grafo auxiliar G'	21
4.3.3. Shortest Path = partición óptima	21
4.4. Pseudocódigo del Split	22

4.5. Inicialización de la Población	23
4.6. Operador de Cruce OX (Order Crossover)	23
4.7. Búsqueda Local (Fase de Educación)	23
4.7.1. LS1 — Relocate (entre rutas y depósitos)	24
4.7.2. LS2 — Swap (intercambio entre rutas)	24
4.7.3. 2-opt (eliminación de cruces)	24
4.8. Distancia BPD y Gestión de Población (el PM)	25
4.9. Pseudocódigo MA PM con explicación línea por línea	26
4.10. Parámetros por defecto y cómo se calculan	28
5. Comparación entre los tres algoritmos	28
6. Resultados de Validación	28
6.1. Gap respecto a BKS (instancias Prodhon)	29
7. Preguntas frecuentes del jurado (con respuestas)	29
8. Ejemplo paso a paso: los tres algoritmos sobre la misma instancia	30

1. El Problema LRP en pocas palabras

Definición

El **Location-Routing Problem (LRP)** responde simultáneamente tres preguntas:

1. **¿Qué depósitos abrir?** (decisión estratégica de localización)
2. **¿A qué depósito asignar cada cliente?** (decisión táctica de asignación)
3. **¿En qué orden visitar los clientes de cada depósito?** (decisión operativa de ruteo)

El objetivo es **minimizar el costo total**: apertura de depósitos + activación de vehículos + distancias recorridas.

Ejemplo

Supongamos la siguiente instancia pequeña con 6 clientes y 2 depósitos:

Nodo	Tipo	x	y	Info adicional
D1	Depósito	10	10	Cap=150, Apertura=500
D2	Depósito	40	40	Cap=150, Apertura=500
C1	Cliente	8	20	Demanda=18
C2	Cliente	15	8	Demanda=15
C3	Cliente	5	15	Demanda=22
C4	Cliente	35	45	Demanda=16
C5	Cliente	45	38	Demanda=14
C6	Cliente	42	48	Demanda=20

Capacidad del vehículo $Q = 60$. Costo fijo de vehículo $F = 100$.

Una solución posible: abrir D1 y D2. Asignar C1,C2,C3 a D1 y C4,C5,C6 a D2.
Con $Q = 60$:

- D1 → Ruta A: [C1, C2] (demanda: 33), Ruta B: [C3] (demanda: 22)
- D2 → Ruta C: [C4, C5] (demanda: 30), Ruta D: [C6] (demanda: 20)

Costo total = 1000 (apertura) + 400 (4 vehículos) + distancias.

2. Algoritmo de Fuerza Bruta (Solución Exacta)

2.1. ¿Qué es y para qué sirve?

El algoritmo de **fuerza bruta** encuentra la solución *óptima garantizada* enumerando todas las configuraciones posibles del problema y eligiendo la de menor costo. Se usa como

referencia de validación: si ALNS y MA|PM coinciden con la fuerza bruta en instancias pequeñas, tenemos confianza de que los metaheurísticos funcionan correctamente.

Nota Clave

La fuerza bruta es **exacta pero lenta**. Para n clientes y m depósitos, el espacio de búsqueda crece como $O(2^m \cdot k^n \cdot 3^{n/k})$. Para $n = 12, m = 2$ tarda 0.2 segundos. Para $n = 20$ tardaría décadas. Por eso solo se usa en instancias pequeñas ($n \leq 12, m \leq 3$).

2.2. Estructura del algoritmo

El algoritmo tiene tres niveles de enumeración:

Nivel 1 — Subconjuntos de depósitos: Para m depósitos, hay $2^m - 1$ subconjuntos no vacíos. Para $m = 2$: $\{D1\}, \{D2\}, \{D1, D2\}$.

Nivel 2 — Asignación de clientes: Si hay k depósitos abiertos, cada uno de los n clientes se puede asignar a cualquiera de los k depósitos. Total: k^n combinaciones.

Nivel 3 — Ruteo óptimo (CVRP exacto): Para cada conjunto de clientes asignado a un depósito, se resuelve el problema de ruteo con capacidad (CVRP) de forma exacta usando dos técnicas encadenadas:

- **Held-Karp DP:** Calcula el costo mínimo de visitar cualquier subconjunto de clientes en el mejor orden posible. Complejidad: $O(2^k \cdot k^2)$.
- **Set-Cover DP:** Divide óptimamente los clientes en rutas que no violen la capacidad Q . Complejidad: $O(3^k)$.

2.3. Held-Karp: cómo funciona

Definición

Held-Karp DP: Define $hk[\text{mask}][\text{last}]$ = mínimo costo de partir del depósito, visitar exactamente los clientes indicados por el bitmask **mask**, y terminar en el cliente **last**.

Recurrencia:

$$hk[S][i] = \min_{j \in S \setminus \{i\}} (hk[S \setminus \{i\}][j] + \text{dist}(c_j, c_i))$$

Costo de ruta completa (ida y vuelta al depósito):

$$\text{route_cost}[S] = F + \min_{i \in S} (hk[S][i] + \text{dist}(c_i, D))$$

Ejemplo

Tenemos el depósito D en $(10,10)$ y 3 clientes: $c_1 = (8, 20)$, $c_2 = (15, 8)$, $c_3 = (5, 15)$. $Q = 60$, $F = 100$. Todos caben en una sola ruta (demanda total = $55 \leq 60$).

Calculamos distancias (en modo entero $\times 100$): $\text{dist}(D, c_1) = \lfloor \sqrt{4 + 100} \cdot 100 \rfloor =$

1019, $dist(D, c_2) = \lfloor \sqrt{25 + 4} \cdot 100 \rfloor = 538$, $dist(D, c_3) = \lfloor \sqrt{25 + 25} \cdot 100 \rfloor = 707$.
 $dist(c_1, c_2) = \lfloor \sqrt{49 + 144} \cdot 100 \rfloor = 1389$, $dist(c_1, c_3) = \lfloor \sqrt{9 + 25} \cdot 100 \rfloor = 583$,
 $dist(c_2, c_3) = \lfloor \sqrt{100 + 49} \cdot 100 \rfloor = 1220$.

Held-Karp base (mask con un solo bit):

$$hk[\{c_1\}][c_1] = dist(D, c_1) = 1019$$

$$hk[\{c_2\}][c_2] = dist(D, c_2) = 538$$

$$hk[\{c_3\}][c_3] = dist(D, c_3) = 707$$

Dos clientes:

$$hk[\{c_1, c_2\}][c_2] = hk[\{c_1\}][c_1] + dist(c_1, c_2) = 1019 + 1389 = 2408$$

$$hk[\{c_1, c_2\}][c_1] = hk[\{c_2\}][c_2] + dist(c_2, c_1) = 538 + 1389 = 1927$$

$$hk[\{c_1, c_3\}][c_3] = hk[\{c_1\}][c_1] + dist(c_1, c_3) = 1019 + 583 = 1602$$

$$hk[\{c_1, c_3\}][c_1] = hk[\{c_3\}][c_3] + dist(c_3, c_1) = 707 + 583 = 1290$$

$$hk[\{c_2, c_3\}][c_3] = hk[\{c_2\}][c_2] + dist(c_2, c_3) = 538 + 1220 = 1758$$

$$hk[\{c_2, c_3\}][c_2] = hk[\{c_3\}][c_3] + dist(c_3, c_2) = 707 + 1220 = 1927$$

Tres clientes ($S = \{c_1, c_2, c_3\}$):

$$\begin{aligned} hk[S][c_3] &= \min(hk[\{c_1, c_2\}][c_1] + dist(c_1, c_3), hk[\{c_1, c_2\}][c_2] + dist(c_2, c_3)) \\ &= \min(1927 + 583, 2408 + 1220) = \min(2510, 3628) = 2510 \quad (\text{llegamos desde } c_1) \end{aligned}$$

Costo de ruta completa (con retorno al depósito):

$$route_cost[\{c_1, c_2, c_3\}] = F + hk[S][c_3] + dist(c_3, D) = 100 + 2510 + 707 = 3317$$

La ruta óptima es: $D \rightarrow c_2 \rightarrow c_1 \rightarrow c_3 \rightarrow D$ con costo 3317.

2.4. Set-Cover DP: cómo funciona

Definición

Set-Cover DP: Una vez que tenemos el $route_cost[S]$ para cada subconjunto S de clientes, buscamos la forma de cubrir **todos** los clientes con el menor costo total:

$$dp[\emptyset] = 0$$

$$dp[M] = \min_{R \subseteq M, R \neq \emptyset, route_cost[R] < \infty} (dp[M \setminus R] + route_cost[R])$$

La respuesta es $dp[\{c_1, c_2, \dots, c_k\}]$.

Ejemplo

Con los mismos 3 clientes y $Q = 60$ (todos caben en una ruta), el Set-Cover DP tiene la respuesta trivial: una sola ruta con los 3 clientes, costo 3317.

Ahora si $Q = 35$ (máximo 2 clientes por ruta por capacidad), las rutas factibles son:

$$route_cost[\{c_1\}] = 100 + dist(D, c_1) + dist(c_1, D) = 100 + 1019 + 1019 = 2138$$

$$route_cost[\{c_2\}] = 100 + 538 + 538 = 1176$$

$$route_cost[\{c_3\}] = 100 + 707 + 707 = 1514$$

$$route_cost[\{c_1, c_2\}] = 100 + \min(1927, 2408) + dist(last, D) = \dots$$

Para $\{c_1, c_2\}$: la mejor llegada es c_1 (desde c_2), entonces costo = $100 + 1927 + dist(c_1, D) = 100 + 1927 + 1019 = 3046$.

El DP decide: ¿conviene $\{c_1, c_2\} + \{c_3\}$ o $\{c_1\} + \{c_2, c_3\}$ o las tres individuales?

$$dp[\{c_1, c_2, c_3\}] = \min \begin{cases} route_cost[\{c_1, c_2\}] + dp[\{c_3\}] = 3046 + 1514 = 4560 \\ route_cost[\{c_1, c_3\}] + dp[\{c_2\}] = \dots + 1176 \\ route_cost[\{c_2, c_3\}] + dp[\{c_1\}] = \dots + 2138 \end{cases}$$

El DP encuentra automáticamente la partición óptima.

2.5. Pseudocódigo completo de la Fuerza Bruta

Algorithm 1 Fuerza Bruta Exacta para LRP

Entrada: Instancia LRP: clientes C , depósitos D , capacidades Q, W_i , costos O_i, F, c_{ij}

Salida: Solución óptima S^* con costo mínimo Z^*

```

1:  $Z^* \leftarrow +\infty, S^* \leftarrow \emptyset$ 
2: for cada subconjunto no vacío  $\mathcal{D}$  de depósitos  $D$  do ▷ Línea 2:  $2^{|\mathcal{D}|} - 1$  iteraciones
3:    $costo\_apertura \leftarrow \sum_{d \in \mathcal{D}} O_d$ 
4:   for cada asignación  $A : C \rightarrow \mathcal{D}$  do ▷ Línea 4:  $|\mathcal{D}|^{|C|}$  iteraciones
5:     if  $\exists d \in \mathcal{D} : \sum_{j: A(j)=d} dem_j > W_d$  then ▷ Línea 5: violar cap. depósito
6:       continuar ▷ Poda por infactibilidad
7:     end if
8:      $costo\_rutas \leftarrow 0$ 
9:     for cada depósito  $d \in \mathcal{D}$  do
10:       $clientes\_d \leftarrow \{j \in C : A(j) = d\}$ 
11:       $costo\_d, rutas\_d \leftarrow CVRP-EXACTO(clientes\_d, d, Q, F)$ 
12:       $costo\_rutas \leftarrow costo\_rutas + costo\_d$ 
13:    end for
14:     $Z \leftarrow costo\_apertura + costo\_rutas$ 
15:    if  $Z < Z^*$  then
16:       $Z^* \leftarrow Z, S^* \leftarrow construir\_solucion(\mathcal{D}, A, rutas)$ 
17:    end if
18:  end for
19: end for
20: return  $S^*, Z^*$ 

```

Algorithm 2 CVRP-Exacto (Held-Karp + Set-Cover DP)**Entrada:** Clientes $K = \{c_0, \dots, c_{k-1}\}$, depósito d , capacidades Q , costo fijo F **Salida:** Costo mínimo y rutas para servir a todos los clientes en K

```

1: Calcular  $sdmnd[mask]$  para todo subconjunto (demanda acumulada)  $\triangleright 2^k$  valores
2: Inicializar  $hk[mask][last] \leftarrow +\infty$  para todo mask, last  $\triangleright$  Tabla Held-Karp
3: for  $i \leftarrow 0$  to  $k - 1$  do
4:    $hk[\{i\}][i] \leftarrow dist(d, c_i)$   $\triangleright$  Caso base: 1 cliente
5: end for
6: for  $S \leftarrow 1$  to  $2^k - 1$  do  $\triangleright$  Por bitmask creciente
7:   if  $sdmnd[S] > Q$  then continuar
8:   end if  $\triangleright$  Poda por capacidad
9:   for  $i \in S$  do  $\triangleright$  Último cliente
10:    for  $j \in S \setminus \{i\}$  do  $\triangleright$  Penúltimo cliente
11:       $c \leftarrow hk[S \setminus \{i\}][j] + dist(c_j, c_i)$ 
12:      if  $c < hk[S][i]$  then  $hk[S][i] \leftarrow c$ , guardar predecesor
13:      end if
14:    end for
15:  end for
16: end for
17: for mask = 1 to  $2^k - 1$  do  $\triangleright$  Costo de ruta completa
18:    $route\_cost[mask] \leftarrow F + \min_{i \in \text{mask}} (hk[mask][i] + dist(c_i, d))$ 
19: end for
20:  $dp[0] \leftarrow 0$ ,  $dp[mask] \leftarrow +\infty$  para todo mask  $\neq 0$   $\triangleright$  Set-Cover DP
21: for mask  $\leftarrow 0$  to  $2^k - 1$  do
22:   for cada subconjunto  $R$  de los clientes no cubiertos por mask do
23:    if  $route\_cost[R] < +\infty$  then
24:       $dp[mask \cup R] \leftarrow \min(dp[mask \cup R], dp[mask] + route\_cost[R])$ 
25:    end if
26:  end for
27: end for
28: return  $dp[2^k - 1]$ , reconstruir rutas desde punteros

```

Glosario de variables del Algoritmo 2:

Variable	Significado
<code>k</code>	Número de clientes asignados a <i>este</i> depósito en la asignación actual. Es el tamaño local del subproblema CVRP; varía entre llamadas.
<code>mask / S</code>	Bitmask (entero) que representa un subconjunto de los k clientes. El bit i vale 1 si el cliente i pertenece al subconjunto. Por ejemplo, con $k = 3$: <code>mask=101</code> $\equiv \{c_0, c_2\}$. Los valores van de 0 ($\emptyset$) a $2^k - 1$ (todos los clientes).
<code>last / i</code>	Índice (0-based) del <i>último cliente visitado</i> en la ruta parcial descrita por <code>mask</code> . <code>hk[mask][last]</code> guarda el costo mínimo de partir del depósito, visitar todos los clientes de <code>mask</code> , y terminar en el cliente <code>last</code> .
<code>j</code>	Índice del <i>penúltimo cliente</i> visitado, es decir, el predecesor de <code>last</code> en la ruta. Se usa en la recurrencia para extender caminos: $hk[S][i] = hk[S \setminus \{i\}][j] + dist(c_j, c_i)$.
<code>sdmnd[mask]</code>	“Subset demand”. Demanda total acumulada de todos los clientes cuyo bit está activo en <code>mask</code> . Precomputado para 2^k valores. Permite verificar $sdmnd[mask] \leq Q$ en $O(1)$.
<code>hk[mask][last]</code>	Tabla principal de Held-Karp. Almacena el costo mínimo del camino: depósito $\rightarrow$ todos los clientes en <code>mask</code> $\rightarrow c_{last}$. Tamaño: $2^k \times k$.
<code>hk_prev[mask][last]</code>	Tabla de punteros de reconstrucción de Held-Karp. Guarda el índice j del penúltimo cliente que llevó al valor óptimo de <code>hk[mask][last]</code> . Se usa al final para trazar el orden exacto de visita.
<code>route_cost[mask]</code>	Costo total de enviar un vehículo desde el depósito a visitar <i>exactamente</i> los clientes en <code>mask</code> y regresar. Incluye el costo fijo F del vehículo. Calculado como $F + \min_i (hk[mask][i] + dist(c_i, D))$.
<code>route_last[mask]</code>	Índice del último cliente de la ruta óptima para <code>mask</code> . Guarda el i que minimizó <code>route_cost[mask]</code> ; necesario para reconstruir el orden de visita.
<code>dp[mask]</code>	Tabla del Set-Cover DP. Costo mínimo de servir a <i>todos</i> los clientes cuyo bit está activo en <code>mask</code> , usando cualquier número de vehículos. <code>dp[$2^k - 1$]</code> es la respuesta final.
<code>dp_prev[mask]</code>	Puntero de reconstrucción del Set-Cover. Guarda el estado <code>mask</code> anterior (antes de añadir la última ruta) que llevó al valor óptimo de <code>dp[mask]</code> .
<code>dp_route[mask]</code>	Puntero de reconstrucción del Set-Cover. Guarda qué subconjunto R (ruta) se añadió para llegar al estado <code>mask</code> óptimamente. Junto con <code>dp_prev</code> , permite reconstruir todas las rutas.
<code>remaining</code>	Complemento de <code>mask</code> respecto al conjunto completo: los clientes que aún no han sido cubiertos. Se usa para iterar solo sobre subconjuntos de clientes pendientes, evitando considerar rutas que ya fueron asignadas.

Explicación línea por línea del Algoritmo 2 (CVRP-Exacto):

L.1 Demanda acumulada por subconjunto. Se precomputan 2^k valores $sdmnd[mask]$, uno por cada posible subconjunto de clientes. Se construyen en orden ascendente de máscara usando el bit menos significativo:

$$sdmnd[mask] = sdmnd[mask \setminus \{lsb\}] + dem_{lsb}$$

Esto permite verificar en $O(1)$ si un subconjunto viola la capacidad Q , evitando recalcular sumas durante el DP. Sin esta precomputación, cada verificación de capacidad costaría $O(k)$.

L.2 Inicialización de la tabla Held-Karp. La tabla $hk[mask][last]$ tiene $2^k \times k$ entradas. Se inicializa todo a $+\infty$ para indicar “aún no alcanzable”. Dimensionar la tabla como arreglo plano 1D ($hk[mask \times k + last]$) evita miles de asignaciones de memoria dinámica.

L.3–4 Casos base. Para cada cliente i individual: el costo de partir del depósito e ir directamente a i es $dist(D, c_i)$. Solo se inicializa si $dem_i \leq Q$ (un solo cliente ya podría violar la capacidad del vehículo en problemas con demandas altas).

L.5–9 Recurrencia principal de Held-Karp. Se itera sobre todos los subconjuntos S en orden creciente de bitmask (garantiza que $S \setminus \{i\}$ ya fue calculado antes de S). Para cada cliente $i \in S$ como último visitado, se busca el mejor penúltimo $j \in S \setminus \{i\}$:

$$hk[S][i] = \min_{j \in S \setminus \{i\}} (hk[S \setminus \{i\}][j] + dist(c_j, c_i))$$

La poda `if sdmnd[S] > Q: continuar` evita calcular rutas infactibles y es la clave del rendimiento: con Q ajustado, la mayoría de subconjuntos se descartan sin entrar al bucle interno.

L.10–12 Costo de ruta completa (ida y vuelta). Para cada subconjunto factible S , se calcula el costo mínimo de la ruta que parte del depósito, visita todos los clientes de S en el mejor orden posible, y regresa al depósito. Incluye el costo fijo F del vehículo:

$$route_cost[S] = F + \min_{i \in S} (hk[S][i] + dist(c_i, D))$$

Este paso “cierra” el ciclo que Held-Karp dejó abierto. Después de este punto, $route_cost[S]$ es el costo exacto y óptimo de enviar un vehículo a visitar exactamente los clientes en S .

L.13 Inicialización del Set-Cover DP. $dp[\emptyset] = 0$ (sin clientes, costo cero). $dp[mask] = +\infty$ para todo $mask \neq 0$ indica “todavía no sabemos cómo cubrir este conjunto de clientes de forma óptima”.

L.14–19 Recurrencia del Set-Cover DP. Para cada estado $mask$ ya resuelto, se considera añadir una nueva ruta R que cubra algunos de los clientes *aún no cubiertos*:

$$dp[mask \cup R] = \min(dp[mask \cup R], dp[mask] + route_cost[R])$$

El truco de iterar subconjuntos con `for route = remaining; route > 0; route = (route-1) & remaining` genera exactamente los $2^{|remaining|}$ subconjuntos no vacíos de los clientes pendientes, sin repeticiones. La complejidad total de este paso es $O(3^k)$ porque cada cliente puede estar en tres estados respecto a cada par (mask, route): (a) ya en mask, (b) en mask y en route, o (c) ni en mask. La condición `route_cost[R] < INF` descarta rutas infactibles (demanda $> Q$).

L.20–21 Resultado y reconstrucción. $dp[2^k - 1]$ es el costo mínimo de cubrir *todos* los k clientes del depósito con cualquier número de vehículos. Si sigue siendo $+\infty$, no existe solución factible (la instancia es infactible para este depósito con este Q). La reconstrucción sigue los punteros dp_prev y dp_route guardados durante el DP para recuperar exactamente qué clientes van en cada ruta y en qué orden.

Nota Clave

¿Por qué dos DPs encadenados y no uno solo? Held-Karp resuelve la pregunta “dado este conjunto fijo de clientes en una ruta, ¿cuál es el mejor orden?” en $O(2^k \cdot k^2)$. Set-Cover resuelve “¿cómo partir todos los clientes en grupos?” en $O(3^k)$. Si intentáramos combinarlos en un solo DP, la tabla necesitaría rastrear simultáneamente el orden dentro de cada ruta *y* la partición entre rutas, lo que llevaría a una explosión combinatoria mucho mayor. Separar los dos problemas es lo que hace factible el algoritmo.

Explicación línea por línea del Algoritmo 1:

- **Línea 1:** Inicializa el mejor costo en infinito (aún no hemos visto nada).
- **Línea 2:** Para $m = 2$ depósitos hay $2^2 - 1 = 3$ subconjuntos: $\{D1\}$, $\{D2\}$, $\{D1, D2\}$.
- **Línea 3:** Suma los costos de apertura del subconjunto actual.
- **Línea 4:** Para $k = 2$ depósitos abiertos y $n = 10$ clientes: $2^{10} = 1024$ asignaciones posibles.
- **Línea 5-7:** Poda temprana: si la demanda total asignada a algún depósito supera su capacidad W_d , esa asignación es infactible. Se descarta sin calcular rutas.
- **Líneas 8-12:** Para cada depósito, resuelve el CVRP exacto con los clientes asignados.
- **Líneas 13-15:** Si el costo total es mejor que el récord, actualiza.

3. ALNS: Adaptive Large Neighborhood Search

3.1. Idea central y motivación

Pregunta frecuente del jurado

¿Por qué usar ALNS si el LRP es tan complejo?

El LRP combinado tiene $2^m \cdot m^n \cdot \prod_d n_d!$ soluciones posibles. Para $n = 20, m = 5$ esto es astronómico. El ALNS navega este espacio de forma inteligente:

1. **Destruye** una parte de la solución actual (quita clientes de sus rutas).
2. **Repara** la solución (reinserta esos clientes de forma óptima).
3. **Aprende** qué operadores funcionan mejor y los usa más (ruleta adaptativa).
4. **Acepta** soluciones peores a veces (Simulated Annealing) para escapar de mínimos locales.

3.2. Representación de la solución

Una solución en ALNS es una tripleta:

$$S = (\mathcal{D}_{open}, \mathcal{R}, U)$$

donde:

- $\mathcal{D}_{open}$: conjunto de depósitos abiertos (ej. $\{D1, D2\}$)
- $\mathcal{R}$: mapa de depósito $\rightarrow$ lista de rutas. Cada ruta es una secuencia de IDs de clientes.
- U : banco de clientes no asignados (vacío en soluciones factibles)

Función objetivo:

$$f(S) = \underbrace{\sum_{d \in \mathcal{D}_{open}} O_d}_{\text{apertura}} + \underbrace{\sum_{r \in \mathcal{R}} \left(F + \sum_{\text{arcos de } r} c_{ij} \right)}_{\text{vehículos y distancia}} + \underbrace{10^5 \cdot |U|}_{\text{penalización huérfanos}}$$

La penalización de 10^5 por cliente no asignado es tan grande que cualquier solución infactible es mucho peor que cualquier factible.

Ejemplo

Solución concreta con la instancia anterior:

- $\mathcal{D}_{open} = \{D1, D2\}$

- $\mathcal{R}[D1] = \{[C1, C2], [C3]\}$

- $\mathcal{R}[D2] = \{[C4, C5], [C6]\}$

- $U = \emptyset$ (todos asignados)

$$f(S) = (500 + 500) + 4 \times 100 + \text{distancias} = 1400 + \text{distancias}$$

3.3. Operadores de Destrucción

Pregunta frecuente del jurado

¿Para qué sirven los operadores de destrucción?

Los destructores **extraen clientes de sus rutas actuales** y los colocan en el banco de no asignados U . Esto crea un “hueco” en la solución que después el operador de reparación debe rellenar mejor. El objetivo es escapar de mínimos locales explorando configuraciones radicalmente diferentes.

Parámetro q : número de clientes a destruir en cada operación (típicamente 10 % – 40 % del total).

3.3.1. 1. Random Removal (Destrucción Aleatoria)

¿Qué hace? Elige q clientes al azar y los extrae de sus rutas.

¿Para qué sirve? Introduce **diversificación pura**. Aunque es “tonto”, evita que la búsqueda se quede atascada explorando siempre la misma zona del espacio de soluciones.

Ejemplo

Solución actual: $D1 \rightarrow [C1, C2, C3]$, $D2 \rightarrow [C4, C5, C6]$. Se sortea $q = 2$. Se eligen aleatoriamente C2 y C5. Resultado tras destrucción:

- $D1 \rightarrow [C1, C3]$ (C2 salió)

- $D2 \rightarrow [C4, C6]$ (C5 salió)

- $U = \{C2, C5\}$

3.3.2. 2. Worst Removal (Destrucción de los Peores)

¿Qué hace? Calcula el **costo marginal** de cada cliente (cuánto costaría sacarlo de su ruta) y extrae los q más caros.

Costo marginal de remover cliente j de la ruta $(\dots, a, j, b, \dots)$:

$$\Delta_j = c_{aj} + c_{jb} - c_{ab}$$

Es decir, lo que ahorramos en distancia si sacamos a j (eliminamos el desvío $a \rightarrow j \rightarrow b$ y lo reemplazamos por la conexión directa $a \rightarrow b$).

¿Para qué sirve? Identifica clientes “costosos” que están mal ubicados en sus rutas actuales y los libera para ser reubicados más eficientemente.

Ejemplo

Ruta: $D1 \rightarrow C1 \rightarrow C3 \rightarrow C2 \rightarrow D1$.

Costo marginal de C3 (ubicado entre C1 y C2):

$$\Delta_{C3} = \text{dist}(C1, C3) + \text{dist}(C3, C2) - \text{dist}(C1, C2) = 583 + 1220 - 1389 = 414$$

Costo marginal de C1 (ubicado entre D1 y C3):

$$\Delta_{C1} = \text{dist}(D1, C1) + \text{dist}(C1, C3) - \text{dist}(D1, C3) = 1019 + 583 - 707 = 895$$

C1 tiene mayor costo marginal $\rightarrow$ Worst Removal lo extrae primero si $q = 1$.

3.3.3. 3. Shaw Removal (Destrucción por Similitud Geográfica)

¿Qué hace? Elige un cliente semilla al azar y extrae los q clientes más **similares** (geográficamente cercanos) a él.

Función de similitud entre clientes i y j :

$$\text{sim}(i, j) = \alpha \cdot c_{ij} + \beta \cdot |\text{dem}_i - \text{dem}_j| + \gamma \cdot \text{mismo_deposito}(i, j)$$

¿Para qué sirve? Si extraemos clientes geográficamente cercanos, el operador de reparación puede reorganizarlos en una ruta más eficiente juntos (clientes cercanos deberían estar en la misma ruta). Esto implementa el principio de **explotar la estructura espacial** del problema.

Ejemplo

Instancia con 6 clientes. Semilla aleatoria: C4 (35,45). Se buscan los 2 más cercanos:

- $\text{dist}(C4, C5) = \text{dist}((35, 45), (45, 38)) = 1220$ (en entero $\times 100$)
- $\text{dist}(C4, C6) = \text{dist}((35, 45), (42, 48)) = 761$

$q = 2$: se extraen C4 (semilla) y C6 (más cercano a C4). Quedan en $U = \{C4, C6\}$. El operador de reparación los reintroducirá probablemente juntos en una ruta, ya que son geográficamente vecinos.

3.3.4. 4. Route Removal (Destrucción de Ruta Completa)

¿Qué hace? Elige una ruta completa al azar y extrae **todos** sus clientes.

¿Para qué sirve? Presiona la **reducción de la flota**. Si una ruta tiene pocos clientes, eliminarla puede ser rentable si sus clientes se absorben en otras rutas existentes, ahorrando el costo fijo F del vehículo eliminado.

Ejemplo

$D1 \rightarrow$ Ruta A: [C1, C2], Ruta B: [C3]. Se elige aleatoriamente la Ruta B. Resultado: $U = \{C3\}$.

Si el operador de reparación logra insertar C3 en la Ruta A (si cabe dentro de Q), se ahorra un vehículo ($F = 100$).

3.3.5. 5. Depot Closing (Cierre de Depósito)

¿Qué hace? Elige un depósito abierto al azar y lo **cierra**: extrae todos sus clientes a U y lo elimina de $\mathcal{D}_{open}$.

¿Para qué sirve? Es el único destructor que actúa sobre las **decisiones estratégicas** de localización. Permite explorar configuraciones con menos depósitos abiertos, potencialmente ahorrando costos fijos de apertura O_i .

Ejemplo

$\mathcal{D}_{open} = \{D1, D2\}$. D1 tiene las rutas [C1,C2] y [C3]. Se cierra D1. Resultado:

- $\mathcal{D}_{open} = \{D2\}$
- $\mathcal{R}[D2] = \{[C4, C5], [C6]\}$ (sin cambios)
- $U = \{C1, C2, C3\}$ (todos los clientes de D1 pasan a no asignados)

El operador de reparación deberá reasignar C1,C2,C3 a D2 o reabrir D1 si es necesario.

3.4. Operadores de Reparación

Los operadores de reparación **reinsertan** los clientes de U en la solución.

3.4.1. 1. Greedy Insertion (Inserción Codiciosa)

¿Qué hace? Para cada cliente en U , encuentra la posición de inserción de menor costo marginal y lo inserta allí.

Costo de inserción de j entre a y b en la ruta de depósito d :

$$\delta(j, a, b, d) = c_{aj} + c_{jb} - c_{ab} + \text{penalización si supera } Q$$

Proceso: Para cada $j \in U$, evalúa todas las posiciones en todas las rutas de todos los depósitos abiertos (y la apertura de nuevas rutas/depósitos). Inserta j en la posición de menor δ .

Nota Clave

¿En qué se diferencia δ (costo de inserción) del costo marginal Δ_j de *worst_removal*?

La fórmula es matemáticamente **idéntica**:

$$\Delta_j = \delta(j, a, b, d) = c_{aj} + c_{jb} - c_{ab}$$

pero el **sentido es opuesto**:

- **Costo marginal** (*worst_removal*): mide cuánto **ahorramos** al *sacar* al cliente j de su posición (a, j, b) . Un valor alto indica que j está mal ubicado y su extracción libera mucho costo.
- **Costo de inserción** (*greedy_repair*): mide cuánto **pagamos** al *meter* al cliente j entre a y b . Un valor bajo indica que j encaja casi sin desviar la ruta.

Son operaciones inversas: si extraemos a j de la posición (a, j, b) con ahorro Δ_j , y luego lo reinsertamos en esa misma posición, pagamos exactamente $\delta = \Delta_j$. El saldo neto es cero. La diferencia semántica existe porque *worst_removal* busca maximizar el ahorro de destrucción, mientras que *greedy_repair* busca minimizar el costo de reinsertación.

Ejemplo

$U = \{C3\}$. Ruta existente en D1: $[C1, C2]$. Demanda actual: 33. $Q = 60$.

Posición 1 (inicio de ruta): $D1 \rightarrow C3 \rightarrow C1 \rightarrow C2$:

$$\delta = \text{dist}(D1, C3) + \text{dist}(C3, C1) - \text{dist}(D1, C1) = 707 + 583 - 1019 = 271$$

Posición 2 (entre C1 y C2): $D1 \rightarrow C1 \rightarrow C3 \rightarrow C2$:

$$\delta = \text{dist}(C1, C3) + \text{dist}(C3, C2) - \text{dist}(C1, C2) = 583 + 1220 - 1389 = 414$$

Posición 3 (al final): $D1 \rightarrow C1 \rightarrow C2 \rightarrow C3$:

$$\delta = \text{dist}(C2, C3) + \text{dist}(C3, D1) - \text{dist}(C2, D1) = 1220 + 707 - 538 = 1389$$

Greedy elige Posición 1 (costo adicional 271, el menor).

3.4.2. 2. Regret-2 Insertion (Inserción con Arrepentimiento)

¿Qué hace? Para cada cliente en U , calcula la diferencia de costo entre su *mejor* y *segunda mejor* posición de inserción. Inserta primero el cliente con mayor “arrepentimiento” (el que más perdería si se deja para después).

$$\text{regret}_2(j) = \delta^{(2)}(j) - \delta^{(1)}(j)$$

¿Por qué es mejor que Greedy? Greedy puede insertar primero un cliente “barato” en cierta posición, pero al hacerlo “bloquea” a otro cliente que en realidad tenía más

urgencia.

Ejemplo

$U = \{C3, C4\}$.

- C3: mejor inserción = 271, segunda mejor = 414. Regret = 143.
- C4: mejor inserción = 150, segunda mejor = 800. Regret = 650.

Greedy insertaría C4 primero (costo 150, el más barato). Regret-2 insertaría C4 primero también en este caso, pero por una razón diferente: si no insertamos C4 ahora, la penalización es enorme (950 de diferencia). El mecanismo de regret evita el “arrepentimiento”.

Caso donde difieren: si C3 fuera muy barato de insertar en múltiples sitios y C4 tiene una sola buena posición que puede ser “ocupada” por otros clientes, Regret-2 prioriza C4.

3.5. Ruleta Adaptativa (el mecanismo de aprendizaje)

Pregunta frecuente del jurado

¿Qué significa “adaptativo” en ALNS?

El ALNS mantiene un **peso** w_i para cada operador. Al inicio todos tienen peso 1. Tras cada uso, el operador recibe una recompensa:

Recompensa	Condición	Valor típico
σ_1	Se encontró nuevo óptimo global	33
σ_2	La solución nueva mejora a la actual	9
σ_3	La solución nueva fue aceptada por SA (aunque peor)	13
σ_4	La solución nueva fue rechazada	1

Cada T iteraciones (“segmento”), los pesos se actualizan:

$$w_{i,\text{nuevo}} = w_{i,\text{antiguo}} \cdot (1 - \rho) + \rho \cdot \frac{\text{score}_i}{\text{usos}_i}$$

donde $\rho \in [0, 1]$ es el factor de reacción (cuánto peso damos a la experiencia reciente). En nuestro código: $\rho = 0,1$, $T = 100$.

Probabilidad de selección (ruleta proporcional):

$$P(\text{elegir operador } i) = \frac{w_i}{\sum_j w_j}$$

Ejemplo

Segmento de 100 iteraciones. Destroyer “Shaw” se usó 30 veces y acumuló score 150 (promedio 5.0 por uso). Destroyer “Random” se usó 40 veces, score 80 (promedio 2.0). Pesos actuales: Shaw=1.8, Random=1.2.

Actualización con $\rho = 0,1$:

$$\begin{aligned}w_{\text{Shaw}}^{\text{nuevo}} &= 1,8 \cdot 0,9 + 0,1 \cdot 5,0 = 1,62 + 0,50 = 2,12 \\w_{\text{Random}}^{\text{nuevo}} &= 1,2 \cdot 0,9 + 0,1 \cdot 2,0 = 1,08 + 0,20 = 1,28\end{aligned}$$

Probabilidades: Shaw se selecciona el 62 % de las veces, Random el 38 %. El algoritmo aprende que Shaw ha sido más efectivo recientemente.

3.6. Simulated Annealing (criterio de aceptación)**Pregunta frecuente del jurado**

¿Por qué aceptar soluciones peores?

La búsqueda local pura siempre acepta soluciones mejores y rechaza las peores, lo que la atrapa en mínimos locales. SA permite “subir colinas” con probabilidad decreciente:

$$P(\text{aceptar } s'|s) = \exp\left(-\frac{f(s') - f(s)}{\mathcal{T}}\right) \quad \text{si } f(s') > f(s)$$

La temperatura $\mathcal{T}$ baja geométricamente en cada iteración:

$$\mathcal{T}_{k+1} = \alpha \cdot \mathcal{T}_k \quad (\alpha \in (0, 1))$$

En nuestro código: $\mathcal{T}_0 = 50000$, $\mathcal{T}_{fin} = 1,0$, $\alpha = 0,998$.

Ejemplo

Solución actual: $f(s) = 11500$. Nueva solución: $f(s') = 11600$ (100 unidades peor). Con temperatura $\mathcal{T} = 500$:

$$P = e^{-100/500} = e^{-0,2} \approx 0,819 \quad (81,9 \% \text{ de aceptación})$$

Con temperatura $\mathcal{T} = 50$ (más avanzado el algoritmo):

$$P = e^{-100/50} = e^{-2} \approx 0,135 \quad (13,5 \% \text{ de aceptación})$$

Al principio el algoritmo acepta soluciones peores fácilmente (exploración). Al final es muy selectivo (explotación).

3.7. Pseudocódigo ALNS con explicación línea por línea

Algorithm 3 ALNS para LRP (detallado)

Entrada: Instancia LRP, parámetros $\mathcal{T}_0, \alpha, \rho, T, \sigma_1, \sigma_2, \sigma_3, \sigma_4, max_iter$

Salida: Mejor solución encontrada s_{best}

```

1:  $s \leftarrow \text{GENERARSOLUCIONINICIAL}$  ▷ Greedy: abrir todos los depósitos
2:  $s_{best} \leftarrow s$  ▷ Mejor global
3:  $w_d^{(i)} \leftarrow 1$  para todo destructor  $i$  ▷ Pesos iniciales iguales
4:  $w_r^{(j)} \leftarrow 1$  para todo reparador  $j$ 
5:  $\mathcal{T} \leftarrow \mathcal{T}_0$  ▷ Temperatura inicial
6: for  $iter = 1$  to  $max\_iter$  do
7:    $i \leftarrow \text{SELECCIONARRULETA}(w_d)$  ▷ Elegir destructor por peso
8:    $j \leftarrow \text{SELECCIONARRULETA}(w_r)$  ▷ Elegir reparador por peso
9:    $s'' \leftarrow \text{DESTRUIR}(s, i)$  ▷ Aplicar destructor  $i$ 
10:   $s' \leftarrow \text{REPARAR}(s'', j)$  ▷ Aplicar reparador  $j$ 
11:  if  $f(s') < f(s_{best})$  then ▷ Nuevo óptimo global
12:     $s \leftarrow s', s_{best} \leftarrow s'$ 
13:     $\sigma \leftarrow \sigma_1$ 
14:  else if  $f(s') < f(s)$  then ▷ Mejora la solución actual
15:     $s \leftarrow s'$ 
16:     $\sigma \leftarrow \sigma_2$ 
17:  else if  $\text{Random}(0, 1) < \exp(-(f(s') - f(s))/\mathcal{T})$  then ▷ Aceptar por SA
18:     $s \leftarrow s'$ 
19:     $\sigma \leftarrow \sigma_3$ 
20:  else
21:     $\sigma \leftarrow \sigma_4$  ▷ Rechazada
22:  end if
23:   $score_i \leftarrow score_i + \sigma$  ▷ Acumular puntaje destructor
24:   $score_j \leftarrow score_j + \sigma$  ▷ Acumular puntaje reparador
25:   $usos_i \leftarrow usos_i + 1, usos_j \leftarrow usos_j + 1$ 
26:   $\mathcal{T} \leftarrow \max(\mathcal{T} \cdot \alpha, \mathcal{T}_{fin})$  ▷ Enfriar
27:  if  $iter \bmod T = 0$  then ▷ Cada segmento
28:     $\text{ACTUALIZARPESOS}(w, score, usos, \rho)$  ▷ Aprendizaje adaptativo
29:  end if
30: end for
31: return  $s_{best}$ 

```

Explicación detallada de cada línea:

L.1 GenerarSolucionInicial: abre todos los depósitos, inserta todos los clientes usando Greedy. Rápido y factible.

L.2 Copia la solución inicial como mejor conocida.

L.3-4 Todos los operadores arrancan con el mismo peso (1.0). La ruleta los usa con igual probabilidad al inicio.

- L.5** La temperatura inicial es alta ($T_0 = 50000$), lo que permite aceptar casi cualquier solución peor.
- L.6** El ciclo principal. Con 5000 iteraciones en nuestro código.
- L.7-8** Ruleta: se sortea un número en $[0, \sum w]$, se elige el operador cuyo peso acumulado alcanza el número sorteado.
- L.9** Primero se aplica el destructor (extrae q clientes a U), luego el reparador (los reinserta). El resultado es s' .
- L.10-20** Clasificación del resultado: 4 casos posibles. Se asigna una recompensa σ al par (destructor, reparador) usado.
- L.21-23** Acumulación de estadísticas para el aprendizaje adaptativo.
- L.24** Enfriamiento geométrico de la temperatura. Con $\alpha = 0,998$ y $T_0 = 50000$: $50000 \times 0,998^{5000} \approx 2,06$.
- L.25-27** Actualización de pesos al final de cada segmento de 100 iteraciones. Los operadores que funcionaron bien reciben mayor peso para ser más seleccionados.

4. MA|PM: Algoritmo Memético con Gestión de Población

4.1. ¿Qué significan MA y PM?

Pregunta frecuente del jurado

¿Qué significa MA|PM?

- **MA (Memetic Algorithm):** Un **Algoritmo Genético** (Genetic Algorithm) combinado con **búsqueda local** intensiva. El término “memético” viene de “meme” (analogía biológica): no solo se hereda el genoma (estructura de rutas), sino también el “conocimiento aprendido” (la búsqueda local pule cada individuo). Es decir: evolución genética + refinamiento individual = Memético.
- **PM (Population Management):** La barra | separa el tipo de algoritmo de su estrategia de gestión de población. “PM” (del inglés *Population Management*) es el mecanismo específico propuesto por Prins et al. (2006) para mantener **diversidad genética**: un individuo nuevo solo entra a la población si es “suficientemente diferente” de los que ya existen, medido por la Distancia de Pares Rotos (BPD).

Nota Clave

El nombre completo en el paper original es MA|PM = **Memetic Algorithm with Population Management**. La distinción es: un GA clásico puede degenerar llenando la población de “clones” (soluciones muy similares). El PM evita esto con el

umbral de diversidad Δ .

4.2. El Cromosoma: Giant Tour + Estado de Depósitos

A diferencia del ALNS, el MA|PM usa **codificación indirecta**:

$$\text{Cromosoma} = \underbrace{(DS_1, DS_2, \dots, DS_m)}_{\text{vector de depósitos}} + \underbrace{(\pi_1, \pi_2, \dots, \pi_{|C|})}_{\text{permutación de clientes (Giant Tour)}}$$

- **DS (Depot State):** Vector binario de longitud m . $DS_i = 1$ si el depósito i está abierto.
- **CS (Client Sequence):** Permutación de los $|C|$ clientes. Esta secuencia se llama **Giant Tour** porque es como un “gran recorrido” que visita a todos los clientes en algún orden, sin indicar dónde empieza/termina cada ruta ni a qué depósito pertenece.

Ejemplo

Con 6 clientes ($C1..C6$) y 2 depósitos:

$$\text{Cromosoma} = \underbrace{(1, 1)}_{\text{DS: D1 y D2 abiertos}} + \underbrace{(C3, C1, C2, C5, C4, C6)}_{\text{CS: Giant Tour}}$$

El Giant Tour no dice nada sobre rutas todavía. Eso lo determinará el algoritmo Split.

4.3. El Algoritmo Split: corazón del MA|PM

Pregunta frecuente del jurado

¿Cómo funciona el algoritmo Split?

El **Split** transforma el Giant Tour en rutas concretas. Es el “decodificador” del cromosoma.

4.3.1. Intuición

Dado el Giant Tour ($C3, C1, C2, C5, C4, C6$), el Split pregunta: ¿dónde “cortamos” esta secuencia para formar rutas válidas que no superen Q y se asignen óptimamente a los depósitos abiertos?

4.3.2. Construcción del grafo auxiliar G'

Se construye un grafo dirigido acíclico con $n+1$ nodos: los nodos $0, 1, \dots, n$ representan los “puntos de corte” en la secuencia. Existe un arco de u a v (con $u < v$) si los clientes $\pi_{u+1}, \dots, \pi_v$ forman una ruta factible (demanda total $\leq Q$). El peso del arco $u \rightarrow v$ es el costo de esa ruta más el costo de asignación al depósito óptimo.

4.3.3. Shortest Path = partición óptima

El **camino más corto** desde el nodo 0 hasta el nodo n en G' define exactamente los cortes y la asignación de rutas a depósitos que minimizan el costo total.

Ejemplo

Giant Tour: $(C3, C1, C2, C5, C4, C6)$. $Q = 35$ (máx 2 clientes por ruta para simplificar). Depósitos abiertos: $D1=(10,10)$, $D2=(40,40)$.

Arcos posibles en G' (rutas de 1 o 2 clientes que cumplen $Q = 35$):

Arcos de un cliente:

- $0 \rightarrow 1$ (ruta $\{C3\}$): demanda 22. Mejor depósito = $D1$ (más cercano). Costo = $O_{D1, \text{sinuevo}} + F + \text{dist}(D1, C3) + \text{dist}(C3, D1)$
- $1 \rightarrow 2$ (ruta $\{C1\}$): demanda 18. $D1$ más cercano.
- ... similarmente para los demás

Arcos de dos clientes:

- $0 \rightarrow 2$ (ruta $\{C3, C1\}$): demanda $22 + 18 = 40 > Q = 35 \rightarrow$ **INFACTIBLE**. No existe arco.
- $1 \rightarrow 3$ (ruta $\{C1, C2\}$): demanda $18 + 15 = 33 \leq 35$. Factible. $D1$ más cercano.
- $3 \rightarrow 5$ (ruta $\{C5, C4\}$): demanda $14 + 16 = 30 \leq 35$. $D2$ más cercano.
- $4 \rightarrow 6$ (ruta $\{C4, C6\}$): demanda $16 + 20 = 36 > Q = 35 \rightarrow$ **INFACTIBLE**.
- $4 \rightarrow 5$ (ruta $\{C4\}$): factible. $5 \rightarrow 6$ (ruta $\{C6\}$): factible.

El algoritmo Bellman-Ford calcula el camino de menor costo de 0 a 6. Supongamos que la solución óptima es:

$0 \rightarrow 3 \rightarrow 5 \rightarrow 6$ (ruta $[0 \rightarrow 3] = \{C3, C1, C2\}$ a $D1$, $[3 \rightarrow 5] = \{C5, C4\}$ a $D2$, $[5 \rightarrow 6] = \{C6\}$ a $D2$).

Espera — $\{C3, C1, C2\}$ tiene demanda $22 + 18 + 15 = 55 > Q = 35$, infactible. En este ejemplo con $Q = 35$ el Split encontraría el camino factible correcto automáticamente.

Con $Q = 60$, el Split podría devolver una ruta única $\{C3, C1, C2\}$ asignada a $D1$ y una sola ruta $\{C5, C4, C6\}$ asignada a $D2$, si eso es lo más barato.

Nota Clave

La **brillantez del Split** es que hace el trabajo duro: dado cualquier Giant Tour, encuentra en tiempo $O(n^2)$ la mejor forma de convertirlo en rutas reales. Esto permite que el cruce genético opere libremente sobre permutaciones de clientes sin preocuparse por factibilidad.

4.4. Pseudocódigo del Split**Algorithm 4** LRP-Split (Bellman-Ford en grafo auxiliar)

Entrada: Giant Tour $\Pi = (\pi_1, \dots, \pi_n)$, depósitos abiertos $\mathcal{D}_{open}$, instancia LRP

Salida: Rutas $\mathcal{R}$ y costo total de routing

```

1:  $d[0] \leftarrow 0, d[v] \leftarrow +\infty$  para  $v = 1, \dots, n$  ▷ DP de Bellman-Ford
2:  $pred[v] \leftarrow -1$  para todo  $v$ 
3: for  $u \leftarrow 0$  to  $n - 1$  do
4:    $dem \leftarrow 0, costo\_ruta \leftarrow 0$ 
5:   for  $v \leftarrow u + 1$  to  $n$  do ▷ Extender la ruta actual
6:      $dem \leftarrow dem + demanda(\pi_v)$ 
7:     if  $dem > Q$  then break
8:     end if ▷ Supera capacidad, no más extensión
9:      $c_{ruta} \leftarrow F + dist\_ruta(\pi_{u+1..v}, \mathcal{D}_{open})$  ▷ Costo + mejor depósito
10:    if  $d[u] + c_{ruta} < d[v]$  then
11:       $d[v] \leftarrow d[u] + c_{ruta}$ 
12:       $pred[v] \leftarrow u$  ▷ Guardar punto de corte
13:    end if
14:  end for
15: end for
16: Reconstruir rutas usando  $pred[]$  desde  $n$  hasta 0
17: return rutas reconstruidas,  $d[n]$ 

```

Explicación línea por línea:

L.1 El nodo 0 es el “inicio” (antes del primer cliente). Su distancia es 0.

L.3-11 Doble bucle: para cada posición de inicio u , extiende la ruta añadiendo clientes $\pi_{u+1}, \pi_{u+2}, \dots$ hasta que la demanda supere Q .

L.5 Acumula la demanda de la ruta $(\pi_{u+1}, \dots, \pi_v)$.

L.6-7 Poda: si agregar π_v haría la demanda superar Q , esa posición de fin no es válida y no tiene sentido seguir extendiendo.

L.8 Calcula el costo de la ruta más el costo del mejor depósito que puede asumir esta ruta sin violar W_i .

L.9-11 Actualiza el DP si este camino es mejor.

L.12 Reconstruye las rutas siguiendo los punteros $pred[]$ de atrás hacia adelante.

4.5. Inicialización de la Población

La población inicial de μ cromosomas se construye así:

1. Una fracción con heurística **Savings** (Clarke-Wright) + ruido estocástico.
2. El resto como permutaciones **aleatorias** del conjunto C .
3. Cada individuo es evaluado con Split y se le aplica **búsqueda local**.

En nuestro código: $\mu = (n + m)/5 + 1$. Para $n = 20, m = 5$: $\mu = 6$.

4.6. Operador de Cruce OX (Order Crossover)

Pregunta frecuente del jurado

¿Por qué OX y no otros cruces?

El OX preserva el **orden relativo** de los clientes, que codifica la estructura espacial de las rutas. Un cliente que en el padre va “antes que” otro mantendrá esa relación en el hijo.

Proceso del OX:

1. Seleccionar dos puntos de corte k_1, k_2 al azar.
2. Copiar el segmento $(\pi_{k_1}, \dots, \pi_{k_2})$ del Padre 1 al Hijo.
3. Completar el Hijo con los clientes restantes del Padre 2, en el orden en que aparecen, saltando los ya presentes.

Ejemplo

Padre 1: $(C3, C1, C2, C5, C4, C6)$. Padre 2: $(C1, C4, C3, C6, C2, C5)$. Cortes en posiciones 2 y 4 (0-indexed):

Padre 1: C3 C1 C2 C5 C4 C6
 $k_1=2$ $k_2=4$

Segmento copiado al hijo: posiciones 2-4 = $(C1, C2, C5)$.

Padre 2 en orden circular desde $k_2 + 1$: $C5$ (ya está), $C1$ (ya está), $C3$ (libre!), $C6$ (libre!), $C2$ (ya está), $C4$ (libre!).

Hijo: $(C3, \mathbf{C1}, \mathbf{C2}, \mathbf{C5}, C6, C4)$.

Los clientes $C1, C2, C5$ vienen del Padre 1 (en ese orden). Los demás $C3, C6, C4$ vienen del orden del Padre 2.

4.7. Búsqueda Local (Fase de Educación)

Después de decodificar con Split, se aplican 3 operadores de mejora:

4.7.1. LS1 — Relocate (entre rutas y depósitos)

¿Qué hace? Intenta mover un cliente de su ruta/depósito actual a otra posición (otra ruta, otro depósito, o nueva ruta). Acepta si reduce el costo.

Ejemplo

Solución actual: $D1 \rightarrow [C1, C2]$, $D2 \rightarrow [C4, C5, C6]$.

Probar mover $C6$ de $D2$ a $D1$:

- Costo de sacar $C6$ de $D2$: $-dist(C5, C6) - dist(C6, D2) + dist(C5, D2)$
- Si $D2$ queda vacía: $-O_{D2}$ (ahorro de apertura)
- Costo de insertar $C6$ al final de $D1$: $+dist(C2, C6) + dist(C6, D1)$

Si el ahorro neto es positivo, se acepta el movimiento.

4.7.2. LS2 — Swap (intercambio entre rutas)

¿Qué hace? Intercambia dos clientes de rutas distintas (o posiciones distintas en la misma ruta). Acepta si reduce el costo.

Ejemplo

$D1 \rightarrow [C1, C2]$, $D2 \rightarrow [C4, C5]$.

Probar $swap(C2, C4)$:

- Nuevo $D1$: $[C1, C4]$. Nuevo $D2$: $[C2, C5]$.
- Calcular cambio en distancias y verificar capacidades.

Si el nuevo costo total es menor, se acepta.

4.7.3. 2-opt (eliminación de cruces)

¿Qué hace? Dentro de una ruta, invierte el orden de un segmento intermedio para eliminar cruces de arcos.

Ejemplo

Ruta: $D1 \rightarrow C1 \rightarrow C3 \rightarrow C2 \rightarrow D1$. Las aristas $C1-C3$ y $C3-C2$ se cruzan geométricamente.

2-opt invierte el segmento $[C3, C2]$: $D1 \rightarrow C1 \rightarrow C2 \rightarrow C3 \rightarrow D1$.

Si $dist(C1, C2) + dist(C3, D1) < dist(C1, C3) + dist(C2, D1)$, el 2-opt mejora la solución.

4.8. Distancia BPD y Gestión de Población (el PM)

Pregunta frecuente del jurado

¿Qué es la BPD y por qué es crucial para MA|PM?

Definición

La **Broken Pairs Distance (BPD)** cuenta cuántos “pares adyacentes” difieren entre dos soluciones. Si en la solución A el cliente i es seguido de j en alguna ruta, ese par (i, j) está “activo”. Si en la solución B ese par no aparece, se dice que está “roto”.

$$BPD(A, B) = \text{número de pares adyacentes en A que no aparecen en B}$$

Ejemplo

Solución A: $D1 \rightarrow [C1 \rightarrow C2 \rightarrow C3]$, $D2 \rightarrow [C4 \rightarrow C5]$. Pares en A: $\{(C1, C2), (C2, C3), (C4, C5)\}$.

Solución B: $D1 \rightarrow [C1 \rightarrow C3]$, $D2 \rightarrow [C4 \rightarrow C2 \rightarrow C5]$. Pares en B: $\{(C1, C3), (C4, C2), (C2, C5)\}$.

Pares de A que no están en B: $(C1, C2)$, $(C2, C3)$, $(C4, C5)$ — los 3 pares “se rompieron”.

$$BPD(A, B) = 3$$

En nuestro código, además se ponderan según el tipo de diferencia:

- Pares adyacentes en misma ruta: penalidad 0 (iguales)
- En misma ruta pero no adyacentes: +1
- En mismo depósito pero ruta diferente: +5
- En depósitos diferentes: +10

El umbral Δ : Un nuevo individuo solo entra a la población si su BPD respecto al más similar de la población es $\geq \Delta$. Si $\Delta = 5$ y el mejor individuo similar tiene $BPD = 3$, el nuevo individuo es rechazado (demasiado parecido).

Nota Clave

El umbral Δ se ajusta dinámicamente: si hay muchos rechazos consecutivos ($NbRej > MaxNbRej$), Δ disminuye en 1, permitiendo individuos más similares. Esto evita que el algoritmo se quede sin nuevos individuos.

4.9. Pseudocódigo MA|PM con explicación línea por línea

Algorithm 5 MA|PM para LRP (Prins et al. 2006)

Entrada: Instancia LRP, parámetros μ , $MaxNbAcc$, $MaxNbNoAdd$, $MaxNbRej$, Δ_{max} , β

Salida: Mejor solución encontrada s_{best}

```

1:  $P \leftarrow \text{GENERARPOBLACION}(\mu, \text{Savings} + \text{Aleatorio})$  ▷ Población inicial
2: for cada  $c \in P$  do
3:    $\text{EVALUAR}(\text{SPLIT}(c))$ 
4:    $\text{BÚSQUEDA}(\text{LOCAL}(c))$  ▷ Fase de educación inicial
5: end for
6:  $s_{best} \leftarrow \arg \min_{c \in P} \text{fitness}(c)$  ▷ Mejor de la población inicial
7:  $NbAcc \leftarrow 0$  ▷ Total de aceptaciones
8: repeat ▷ Loop externo
9:    $\Delta \leftarrow \Delta_{max}$ ,  $NbNoAdd \leftarrow 0$  ▷ Reset parcial por iteración externa
10:  repeat ▷ Loop interno
11:     $P_a \leftarrow \text{SELECCIONAR}(\text{PADREA}(P, \beta))$  ▷ Torneo: toma el mejor entre  $\beta$  candidatos
12:     $P_b \leftarrow \text{SELECCIONAR}(\text{PADREB}(P, P_a))$  ▷ Padre diferente, aleatorio
13:     $H \leftarrow \text{OX-CROSSOVER}(P_a, P_b)$  ▷ Cruce de orden en el Giant Tour
14:     $\text{EVALUAR}(\text{SPLIT}(H))$  ▷ Decodificar: Giant Tour  $\rightarrow$  rutas
15:     $\text{BÚSQUEDA}(\text{LOCAL}(H))$  ▷ Educación: LS1 con prob  $p_1$ , LS2 con prob  $p_2$ 
16:    if  $\text{fitness}(H) < \text{fitness}(s_{best})$  then ▷ Nuevo óptimo global
17:       $s_{best} \leftarrow H$ ,  $\Delta \leftarrow \Delta_{max}$ 
18:    end if
19:     $d_{pop} \leftarrow \text{BPD}(\text{MIN}(H, P))$  ▷ Distancia mínima a la población
20:    if  $d_{pop} < \Delta$  then ▷ Demasiado similar
21:       $NbRej \leftarrow NbRej + 1$ ,  $NbNoAdd \leftarrow NbNoAdd + 1$ 
22:    else ▷ Suficientemente diverso
23:       $NbRej \leftarrow 0$ ,  $NbAcc \leftarrow NbAcc + 1$ 
24:       $\text{INSERTAR}(\text{ENPOBLACION}(P, H))$  ▷ Reemplaza al peor
25:    end if
26:    if  $NbRej > \text{MaxNbRej}$  then
27:       $\Delta \leftarrow \max(1, \Delta - 1)$  ▷ Relajar umbral de diversidad
28:    end if
29:  until  $NbNoAdd > \text{MaxNbNoAdd}$  o  $NbAcc > \text{MaxNbAcc}$  ▷ Condición OR
30:     $\text{INSERTAR}(\text{ENPOBLACION}(P, s_{best}))$  ▷ Reinsertar elite
31:     $\text{REFRESCAR}(\text{POBLACION}(P))$  ▷ Sustituir los peores
32: until  $NbAcc > \text{MaxNbAcc}$  ▷ Loop externo
33: return  $s_{best}$ 

```

Explicación detallada:

L.1 Construye la población inicial. Parte de los individuos vienen de heurísticas constructivas, parte son aleatorios.

L.2-4 Evalúa cada individuo: primero Split (obtener rutas del Giant Tour) y luego búsqueda local (mejorar las rutas). Esta “educación inicial” da calidad desde el arranque.

- L.5** Guarda al mejor individuo como solución élite global.
- L.6** Contador global de aceptaciones (no se resetea entre iteraciones externas).
- L.7** Loop externo: continúa hasta que el total de aceptaciones supere $MaxNbAcc$.
- L.8** Reinicia Δ al valor máximo y el contador de no-adiciones.
- L.9** Loop interno: genera y evalúa nuevos hijos.
- L.10** SeleccionarPadreA: torneo de β candidatos, gana el de menor costo. $\beta = NbIndiv/3$.
- L.11** SeleccionarPadreB: aleatorio entre los restantes.
- L.12** Cruce OX sobre los vectores CS (Giant Tour). Los vectores DS también se cruzan.
- L.13** Split convierte el Giant Tour del hijo en rutas explícitas y calcula su fitness.
- L.14** Con probabilidad $p_1 = 0,5$ se aplica LS1 (Relocate), con probabilidad $p_2 = 0,7$ se aplica LS2 (Swap), con el resto ninguno.
- L.15-17** Si el hijo supera el récord global, actualizar y resetear Δ .
- L.18** Calcula la BPD mínima del hijo respecto a todos los individuos actuales de la población.
- L.19-21** Rechazo: el hijo es demasiado parecido a alguien de la población. Se incrementan contadores pero no se modifica la población.
- L.22-24** Aceptación: el hijo es suficientemente diverso. Se resetea el contador de rechazos consecutivos, se acepta ($NbAcc++$) y se inserta en la población (reemplazando al peor).
- L.25-27** Si hay demasiados rechazos consecutivos ($NbRej > MaxNbRej$), el umbral Δ se reduce en 1 para ser más permisivo con la diversidad.
- L.28 Condición clave (OR):** el loop interno sale cuando *o bien* hay demasiada estagnación ($NbNoAdd > MaxNbNoAdd$) *o bien* ya se aceptaron suficientes individuos ($NbAcc > MaxNbAcc$).
- L.29** Reinserta al mejor conocido en la población (garantiza que la élite no se pierda en el refresco).
- L.30** RefrescarPoblacion: genera nuevos individuos para reemplazar a los peores, asegurando variedad para la siguiente iteración externa.

4.10. Parámetros por defecto y cómo se calculan

Para una instancia con n clientes y m depósitos, los parámetros se calculan automáticamente:

Parámetro	Fórmula	Ejemplo ($n = 20, m = 5$)
$\mu = \text{NbIndiv}$	$(n + m)/5 + 1$	6 individuos
MaxNbAcc	$(n + m) \times 10$	250
MaxNbNoAdd	$(n + m) \times 2,5$	62
MaxNbRej	μ	6
Δ_{max}	$(n + m)/10 + 10$	12
β	$\text{máx}(1, \mu/3)$	2
p_1	0.5	0.5
p_2	0.7	0.7
α_{pen}	1000.0	1000

5. Comparación entre los tres algoritmos

Aspecto	Fuerza Bruta	ALNS	MA PM
Tipo	Exacto	Metaheurístico	Metaheurístico
Garantía	Óptimo global	No garantizada	No garantizada
Escalabilidad	$n \leq 12$	$n \leq 200+$	$n \leq 200+$
Representación	Implícita	Rutas explícitas	Giant Tour (indirecto)
Diversificación	Enumeración completa	Destruyores + SA	Cruce OX + BPD
Intensificación	DP exacto	Reparadores codiciosos	Búsqueda local (LS1/LS2/2-opt)
Aprendizaje	N/A	Ruleta adaptativa de pesos	N/A (evolución poblacional)
Tiempo ($n = 20$)	Intratable	$\sim 1s$	$\sim 0.3s$

6. Resultados de Validación

Se ejecutaron los tres algoritmos sobre las instancias pequeñas creadas ad-hoc:

Instancia	n	m	BF (óptimo)	ALNS gap %	MA PM gap %
coord10-2-1.dat	10	2	11576	0.00 %	0.00 %
coord10-2-2.dat	10	2	15912	0.00 %	0.33 %
coord12-2-1.dat	12	2	15123	0.00 %	0.00 %

Nota Clave

El gap 0.00 % en la mayoría de instancias indica que ambos metaheurísticos **encuentran el óptimo global** en instancias pequeñas. El gap 0.33 % en coord10-2-2.dat con MA|PM es normal: el algoritmo genético tiene componente estocástico y en esta corrida particular no llegó al óptimo exacto (con más iteraciones o diferente semilla probablemente lo encontraría).

6.1. Gap respecto a BKS (instancias Prodhon)

Para las instancias reales de benchmarking, el gap se calcula respecto a la mejor solución conocida (BKS):

$$Gap(\%) = \frac{Z_{alg} - Z_{BKS}}{Z_{BKS}} \times 100$$

Un gap menor indica mejor calidad. Los metaheurísticos del estado del arte logran gaps entre 0 % y 5 % en las instancias Prodhon.

7. Preguntas frecuentes del jurado (con respuestas)**Pregunta frecuente del jurado**

¿Por qué el ALNS tiene una solución inicial tan simple (All-Open)?

Porque el ALNS no necesita una buena solución inicial: sus operadores de destrucción/reparación y el SA explorarán y mejorarán desde cualquier punto de arranque. Una inicialización simple reduce el tiempo de setup. El operador `depot_closing` es quien “limpiará” los depósitos redundantes durante la optimización.

Pregunta frecuente del jurado

¿Qué diferencia hay entre Greedy y Regret-2?

Greedy es miope: siempre elige la inserción más barata en el momento. Regret-2 es previsor: inserta primero al cliente que más perdería si espera (el que tiene mayor diferencia entre su mejor y segunda mejor posición). En problemas donde los clientes “compiten” por las mismas posiciones, Regret-2 produce soluciones iniciales de mejor calidad.

Pregunta frecuente del jurado

¿Qué pasa si el umbral Δ llega a 1?

Con $\Delta = 1$, prácticamente todo individuo es aceptado (cualquier diferencia de BPD ≥ 1 basta). El algoritmo entra en modo de “exploración masiva”. Esto ocurre cuando la población se ha vuelto muy homogénea y el sistema necesita desesperadamente diversidad para no estancarse.

Pregunta frecuente del jurado

¿Por qué el Split usa Bellman-Ford y no Dijkstra?

El grafo auxiliar es un DAG (grafo acíclico dirigido) con pesos no negativos. Bellman-Ford funciona para cualquier tipo de grafo, pero en un DAG con pesos no negativos se puede usar simplemente **programación dinámica en orden topológico** (que es lo que hace el Split). Dijkstra también funcionaría. Lo importante es que el orden topológico del DAG es simplemente el orden numérico de los nodos $0, 1, \dots, n$, lo que hace al algoritmo muy eficiente.

Pregunta frecuente del jurado

¿Por qué el MA|PM no garantiza el óptimo global?

Porque: (1) el cruce OX y la búsqueda local son heurísticos, no sistemáticos; (2) la gestión de población con BPD puede rechazar individuos buenos por ser similares a otros ya en la población; (3) no hay mecanismo de “vuelta atrás” como en branch-and-bound. Sin embargo, en la práctica, para instancias de hasta 200 clientes, MA|PM obtiene resultados muy próximos al óptimo (gaps $< 2\%$).

8. Ejemplo paso a paso: los tres algoritmos sobre la misma instancia

En esta sección resolvemos **exactamente la misma instancia** con Fuerza Bruta, ALNS y MA|PM, trazando cada paso para ver en qué difieren en enfoque, esfuerzo computacional y calidad de solución.

Instancia de ejemplo

Todos los nodos están sobre el eje x ; las distancias son diferencias absolutas.

Nodo	Posición x	Tipo	Demanda
D_1	0	Depósito	—
D_2	6	Depósito	—
C_1	1	Cliente	1
C_2	3	Cliente	1
C_3	5	Cliente	1

Parámetros: $n = 3$ clientes, $m = 2$ depósitos, capacidad de vehículo $Q = 2$, costo fijo de ruta $F = 2$, costo de apertura de depósito $O = 3$ (igual para ambos).

Matriz de distancias $d[i][j] = |x_i - x_j|$:

	D_1	C_1	C_2	C_3	D_2
D_1	0	1	3	5	6
C_1	1	0	2	4	5
C_2	3	2	0	2	3
C_3	5	4	2	0	1
D_2	6	5	3	1	0

Nota Clave

La **solución óptima global** tiene costo **18**: abrir D_1 y D_2 ; ruta $D_1 \rightarrow C_1 \rightarrow D_1$ (coste $1 + 1 + F = 4$) y ruta $D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2$ (coste $3 + 2 + 1 + F = 8$). Total $= O_1 + O_2 + 4 + 8 = 18$.

Algoritmo 1 — Fuerza Bruta (solución exacta)

La fuerza bruta recorre los $2^m - 1 = 3$ subconjuntos no vacíos de depósitos. Para cada subconjunto abre esos depósitos y calcula el costo mínimo de servir a todos los clientes usando dos DP's anidados.

Caso A — Solo D_1 abierto

Paso 1: Held-Karp para cada ruta factible (demanda $\leq Q = 2$):

Subconjunto de ruta	Mejor orden (Held-Karp)	Coste viaje + F
$\{C_1\}$	$D_1 \rightarrow C_1 \rightarrow D_1$	$1 + 1 + 2 = 4$
$\{C_2\}$	$D_1 \rightarrow C_2 \rightarrow D_1$	$3 + 3 + 2 = 8$
$\{C_3\}$	$D_1 \rightarrow C_3 \rightarrow D_1$	$5 + 5 + 2 = 12$
$\{C_1, C_2\}$	$D_1 \rightarrow C_1 \rightarrow C_2 \rightarrow D_1$	$1 + 2 + 3 + 2 = 8$
$\{C_1, C_3\}$	$D_1 \rightarrow C_1 \rightarrow C_3 \rightarrow D_1$	$1 + 4 + 5 + 2 = 12$
$\{C_2, C_3\}$	$D_1 \rightarrow C_2 \rightarrow C_3 \rightarrow D_1$	$3 + 2 + 5 + 2 = 12$

($\{C_1, C_2, C_3\}$ tiene demanda $3 > Q = 2$, se descarta.)

Paso 2: Set-Cover DP (máscara: $C_1 = \text{bit } 0$, $C_2 = \text{bit } 1$, $C_3 = \text{bit } 2$):

Máscara	Subconjunto	$dp[\text{máscara}]$ y razonamiento
000	$\emptyset$	0
001	$\{C_1\}$	$0 + 4 = 4$
010	$\{C_2\}$	$0 + 8 = 8$
100	$\{C_3\}$	$0 + 12 = 12$
011	$\{C_1, C_2\}$	$\min(0 + 8, 4 + 8, 8 + 4) = 8$ mejor: ruta $\{C_1, C_2\}$ directa
101	$\{C_1, C_3\}$	$\min(0 + 12, 4 + 12, 12 + 4) = 12$
110	$\{C_2, C_3\}$	$\min(0 + 12, 8 + 12, 12 + 8) = 12$
111	$\{C_1, C_2, C_3\}$	$\min(\underbrace{dp[110] + 4}_{=16}, dp[101] + 8, dp[011] + 12) = 16$ mejor: ruta $\{C_3\}$ sobre $dp[011] = 8$, o bien ruta $\{C_1\}$ sobre $dp[110] = 12$

Costo total caso A: $O(D_1) + dp[111] = 3 + 16 = \mathbf{19}$. Partición óptima: $\{C_1, C_2\}$ (ruta coste 8) + $\{C_3\}$ (ruta coste 8).

Caso B — Solo D_2 abierto

Held-Karp desde D_2 :

Subconjunto	Mejor orden	Coste viaje + F
$\{C_1\}$	$D_2 \rightarrow C_1 \rightarrow D_2$	$5 + 5 + 2 = 12$
$\{C_2\}$	$D_2 \rightarrow C_2 \rightarrow D_2$	$3 + 3 + 2 = 8$
$\{C_3\}$	$D_2 \rightarrow C_3 \rightarrow D_2$	$1 + 1 + 2 = 4$
$\{C_1, C_2\}$	$D_2 \rightarrow C_2 \rightarrow C_1 \rightarrow D_2$	$3 + 2 + 5 + 2 = 12$
$\{C_1, C_3\}$	$D_2 \rightarrow C_3 \rightarrow C_1 \rightarrow D_2$	$1 + 4 + 5 + 2 = 12$
$\{C_2, C_3\}$	$D_2 \rightarrow C_3 \rightarrow C_2 \rightarrow D_2$	$1 + 2 + 3 + 2 = 8$

Set-Cover DP (análogo al caso A, valores cambian por los costos de ruta desde D_2): $dp[111] = 16$, logrando la partición $\{C_1, C_2\}$ (costo 12) + $\{C_3\}$ (costo 4).

Costo total caso B: $O(D_2) + 16 = 3 + 16 = \mathbf{19}$.

Caso C — Ambos depósitos $\{D_1, D_2\}$

Se prueban todas las formas de asignar los 3 clientes entre los dos depósitos ($2^3 - 2 = 6$ asignaciones no degeneradas):

Asignación	Rutas D_1	Rutas D_2	Costo total
$D_1 : \{C_1\}, D_2 : \{C_2, C_3\}$	4	8	$3 + 3 + 4 + 8 = \mathbf{18}$
$D_1 : \{C_2\}, D_2 : \{C_1, C_3\}$	8	12	$3 + 3 + 8 + 12 = 26$
$D_1 : \{C_3\}, D_2 : \{C_1, C_2\}$	12	12	$3 + 3 + 12 + 12 = 30$
$D_1 : \{C_1, C_2\}, D_2 : \{C_3\}$	8	4	$3 + 3 + 8 + 4 = \mathbf{18}$
$D_1 : \{C_1, C_3\}, D_2 : \{C_2\}$	12	8	$3 + 3 + 12 + 8 = 26$
$D_1 : \{C_2, C_3\}, D_2 : \{C_1\}$	12	12	$3 + 3 + 12 + 12 = 30$

Costo total caso C: mínimo = **18**.

Resultado de la Fuerza Bruta

Caso	Costo mínimo
Solo D_1	19
Solo D_2	19
D_1 y D_2	18 ← óptimo global

Solución óptima confirmada: abrir D_1 y D_2 ; ruta $D_1 \rightarrow C_1 \rightarrow D_1$ (coste 4) y ruta $D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2$ (coste 8). Costo total = **18**.

Algoritmo 2 — ALNS

El ALNS arranca con todos los depósitos abiertos y una solución greedy, luego aplica destrucción/reparación guiada por Simulated Annealing.

Solución inicial S_0

Para ilustrar la mejora, asumimos que la asignación greedy inicial coloca C_3 en D_1 (por ejemplo, por orden de procesamiento):

- D_1 abierto, ruta $[C_1, C_3]: D_1 \rightarrow C_1 \rightarrow C_3 \rightarrow D_1 + F = 1 + 4 + 5 + 2 = 12$
- D_2 abierto, ruta $[C_2]: D_2 \rightarrow C_2 \rightarrow D_2 + F = 3 + 3 + 2 = 8$

$$\text{Costo}(S_0) = O_1 + O_2 + 12 + 8 = 3 + 3 + 12 + 8 = \mathbf{26}$$

Parámetros SA: $T_0 = 1000$, $\alpha = 0,95$. Mejor solución guardada: $S^* = S_0$, costo 26.

Iteración 1: random_removal + greedy_repair

Destrucción — se elige aleatoriamente $q = 1$ cliente a eliminar; sale C_3 .

- Estado temporal: $D_1 : [C_1], D_2 : [C_2], no_asig = \{C_3\}$

Reparación — greedy_repair evalúa el delta de insertar C_3 en cada posición posible:

Opción de inserción	Costo incremental Δ	Detalle
Insertar C_3 tras C_1 en D_1	+8	Ruta pasa de coste 4 a 12
Insertar C_3 tras C_2 en D_2	0	Viaje $D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2 = 3 + 2 + 1 = 6$, igual q
Nueva ruta C_3 solo en D_2	+4	$D_2 \rightarrow C_3 \rightarrow D_2 + F = 4$

La inserción con $\Delta = 0$ (tras C_2 en D_2) es la más barata. Se aplica.

S_1 : $D_1 : [C_1]$, $D_2 : [C_2, C_3]$ Costo(S_1) = $3 + 3 + 4 + 8 = 18$

$\Delta E = 18 - 26 = -8 < 0 \Rightarrow$ aceptación garantizada (mejora). S_1 pasa a ser la nueva mejor solución S^* . Se incrementan los pesos de **random_removal** y **greedy_repair** con σ_1 (recompensa por nuevo mejor).

Iteración 2: **worst_removal** + **regret2_repair**

Destrucción — **worst_removal** elimina el cliente que más contribuye al costo total. Ruta $D_1 : [C_1]$ tiene coste 4 por 1 cliente; ruta $D_2 : [C_2, C_3]$ tiene coste 8 por 2 clientes. Ambas cuestan 4 por cliente; por desempate se elige C_1 .

Estado temporal: $D_1 : []$, $D_2 : [C_2, C_3]$, $no_asig = \{C_1\}$.

Reparación — **regret2_repair** calcula el arrepentimiento de C_1 :

- Mejor inserción: nueva ruta C_1 en D_1 ($D_1 \rightarrow C_1 \rightarrow D_1 + F = 4$), $\Delta_1 = +4$.
- Segunda mejor: nueva ruta C_1 en D_2 ($D_2 \rightarrow C_1 \rightarrow D_2 + F = 12$), $\Delta_2 = +12$. (No cabe en la ruta $[C_2, C_3]$ porque $Q = 2$ está lleno.)
- Arrepentimiento = $\Delta_2 - \Delta_1 = 8$. Alto $\Rightarrow$ insertar inmediatamente.

$S_2 = S_1$: $D_1 : [C_1]$, $D_2 : [C_2, C_3]$, costo **18**. $\Delta E = 0$. Aceptado. Pesos de **worst_removal** y **regret2_repair** reciben σ_3 .

Iteración 3: **depot_closing** (operador de cierre)

El operador intenta cerrar un depósito para ahorrar O .

Intento de cerrar D_1 : D_1 tiene solo la ruta $[C_1]$.

- Ahorro por cierre: $-O(D_1) - \text{ruta}(C_1) = -3 - 4 = -7$.
- C_1 debe reasignarse a D_2 . La ruta $[C_2, C_3]$ está llena ($Q = 2$) $\Rightarrow$ nueva ruta en D_2 .
- Costo nueva ruta: $D_2 \rightarrow C_1 \rightarrow D_2 + F = 5 + 5 + 2 = 12$.
- $\Delta E = -7 + 12 = +5$.

S_3 : solo D_2 abierto, rutas $[C_2, C_3]$ y $[C_1]$. Costo = $3 + 8 + 12 = 23$.

Temperatura actual $T_3 = 1000 \cdot 0,95^2 \approx 902$. $P(\text{aceptar}) = e^{-5/902} \approx 0,994$. Casi siempre aceptado en SA caliente $\Rightarrow$ exploración.

Nota Clave

Una vez que D_1 se cierra, las reparaciones **greedy_repair** y **regret2_repair** **solo pueden usar depósitos abiertos**. Si la solución queda atrapada en “ D_2 único”, el costo mínimo alcanzable es 19 (no 18). Esta es la limitación conocida del ALNS en LRP: puede cerrar depósitos pero no reabrirlos.

En la práctica, si T todavía es alta, el SA puede aceptar esta exploración y el algoritmo escapa más tarde gracias a otros operadores. La mejor solución *recordada* $S^* = 18$ nunca se pierde aunque las iteraciones exploren peor.

Resultado del ALNS

Tras enfriamiento completo ($T \rightarrow 0$), el algoritmo retorna $S^* = \{D_1 : [C_1], D_2 : [C_2, C_3]\}$, costo **18**.

Algoritmo 3 — MA|PM (Algoritmo Memético)

El MA|PM trabaja con una **población de cromosomas**, cada uno formado por el estado de depósitos (DS) y un tour gigante (CS).

Población inicial (2 cromosomas para este ejemplo)

Cromosoma	DS	CS	Descripción
Chr1	[1, 0]	$[C_2, C_3, C_1]$	D_1 abierto; D_2 cerrado
Chr2	[0, 1]	$[C_1, C_2, C_3]$	D_1 cerrado; D_2 abierto

$DS[i] = 1$ significa que el depósito i está habilitado. CS es el *tour gigante*: la permutación de clientes sin indicar rutas (el Split las determinará).

Split de Chr1 — DS= [1, 0], CS= $[C_2, C_3, C_1]$

Solo D_1 disponible. Calculamos $dp[0..3]$ donde $dp[k]$ = costo mínimo de servir los primeros k clientes del tour. $dp[0] = 0$. Para cada posición inicial i probamos segmentos $[i+1 .. j]$ con demanda total $\leq Q = 2$.

Segmento	Clientes	Mejor depósito y ruta	Costo ruta	
[1.,1]	$\{C_2\}$	$D_1 \rightarrow C_2 \rightarrow D_1 + F = 3 + 3 + 2$	8	
[1.,2]	$\{C_2, C_3\}$	$D_1 \rightarrow C_2 \rightarrow C_3 \rightarrow D_1 + F = 3 + 2 + 5 + 2$	12	
[2.,2]	$\{C_3\}$	$D_1 \rightarrow C_3 \rightarrow D_1 + F = 5 + 5 + 2$	12	$dp[2] = \min(12, 8 + 12)$
[2.,3]	$\{C_3, C_1\}$	$D_1 \rightarrow C_3 \rightarrow C_1 \rightarrow D_1 + F = 5 + 4 + 1 + 2$	12	$dp[3] = 0 +$
[3.,3]	$\{C_1\}$	$D_1 \rightarrow C_1 \rightarrow D_1 + F = 1 + 1 + 2$	4	$dp[3] = \min$

Reconstrucción hacia atrás: $dp[3] = 16$ desde $dp[2] + 4 \Rightarrow$ segmento $[3.,3] = \{C_1\}$ en D_1 . $dp[2] = 12$ desde $dp[0] + 12 \Rightarrow$ segmento $[1.,2] = \{C_2, C_3\}$ en D_1 .

Rutas Chr1: $D_1 \rightarrow C_2 \rightarrow C_3 \rightarrow D_1$ (coste 12) y $D_1 \rightarrow C_1 \rightarrow D_1$ (coste 4). Costo total = $O(D_1) + 12 + 4 = 3 + 16 = 19$.

Split de Chr2 — **DS** = $[0, 1]$, **CS** = $[C_1, C_2, C_3]$

Solo D_2 disponible.

Segmento	Clientes	Mejor depósito y ruta	Costo ruta
$[1.,1]$	$\{C_1\}$	$D_2 \rightarrow C_1 \rightarrow D_2 + F = 5 + 5 + 2$	12
$[1.,2]$	$\{C_1, C_2\}$	$D_2 \rightarrow C_1 \rightarrow C_2 \rightarrow D_2 + F = 5 + 2 + 3 + 2$	12
$[2.,2]$	$\{C_2\}$	$D_2 \rightarrow C_2 \rightarrow D_2 + F = 3 + 3 + 2$	8
$[2.,3]$	$\{C_2, C_3\}$	$D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2 + F = 3 + 2 + 1 + 2$	8
$[3.,3]$	$\{C_3\}$	$D_2 \rightarrow C_3 \rightarrow D_2 + F = 1 + 1 + 2$	4

Rutas Chr2: $D_2 \rightarrow C_1 \rightarrow C_2 \rightarrow D_2$ (coste 12) y $D_2 \rightarrow C_3 \rightarrow D_2$ (coste 4). Costo total = $O(D_2) + 12 + 4 = 3 + 16 = 19$.

Búsqueda local LS1 sobre Chr1

Estado: $D_1 : \{C_2, C_3\}, \{C_1\}$. Costo = 19. Intentamos or-opt (mover 1 cliente):

Movimiento	Nuevas rutas	Costo total
Mover C_2 de $\{C_2, C_3\}$ a $\{C_1\}$	$\{C_3\}, \{C_1, C_2\}$	$3 + 12 + 8 = 23$
Mover C_3 de $\{C_2, C_3\}$ a $\{C_1\}$	$\{C_2\}, \{C_1, C_3\}$	$3 + 8 + 12 = 23$

Ningún movimiento mejora. **Chr1 es localmente óptimo en 19.**

Búsqueda local LS1 sobre Chr2

Estado: $D_2 : \{C_1, C_2\}, \{C_3\}$. Análisis simétrico. **Chr2 es localmente óptimo en 19.**

Cruce OX entre Chr1 y Chr2

Seleccionamos Chr1 y Chr2 como padres.

Cruce de DS (uniforme): tomamos D_1 de Chr1 (= 1) y D_2 de Chr2 (= 1). $\Rightarrow$ $DS_{hijo} = [1, 1]$ (ambos depósitos abiertos).

Cruce OX de CS, punto de corte en posición $[0..0]$:

1. Copiar $P_1[0] = C_2$ al hijo en posición 0.
2. Completar con el orden de $P_2 = [C_1, C_2, C_3]$, omitiendo C_2 : quedan $[C_1, C_3]$.
3. Hijo $CS = [C_2, C_1, C_3]$.

Split del Hijo — DS= [1, 1], CS= [C₂, C₁, C₃]

Ambos depósitos disponibles. En cada segmento elegimos el depósito de menor costo.

Segmento	Clientes	Mejor depósito y ruta	Costo ruta
[1.,1]	{C ₂ }	D ₁ : 3 + 3 + 2 = 8; D ₂ : 3 + 3 + 2 = 8 (empate)	8
[1.,2]	{C ₂ , C ₁ }	D ₁ : 3 + 2 + 1 + 2 = 8; D ₂ : 3 + 2 + 5 + 2 = 12	8
[2.,2]	{C ₁ }	D ₁ : 1 + 1 + 2 = 4; D ₂ : 5 + 5 + 2 = 12	4
[2.,3]	{C ₁ , C ₃ }	D ₁ : 1 + 4 + 5 + 2 = 12; D ₂ : 5 + 4 + 1 + 2 = 12	12
[3.,3]	{C ₃ }	D ₁ : 5 + 5 + 2 = 12; D ₂ : 1 + 1 + 2 = 4	4

Reconstrucción: $dp[3] = 12$ desde $dp[2] + 4 \Rightarrow$ segmento $[3.,3] = \{C_3\}$ en D_2 . $dp[2] = 8$ desde $dp[0] + 8 \Rightarrow$ segmento $[1.,2] = \{C_2, C_1\}$ en D_1 .

Rutas del hijo: $D_1 \rightarrow C_2 \rightarrow C_1 \rightarrow D_1 + F$ (coste 8) y $D_2 \rightarrow C_3 \rightarrow D_2 + F$ (coste 4).
Costo total = $O(D_1) + O(D_2) + 8 + 4 = 3 + 3 + 8 + 4 = \mathbf{18}$ ✓

Verificación BPD (gestión de diversidad)

El hijo solo se incorpora a la población si es suficientemente diferente de **todos** los individuos existentes. La distancia BPD cuenta los pares adyacentes que difieren.

Individuo	Rutas	Pares adyacentes (clave simétrica)
Chr1	$D_1 : [C_2, C_3], [C_1]$	$\{(C_2, C_3)\}$
Chr2	$D_2 : [C_1, C_2], [C_3]$	$\{(C_1, C_2)\}$
Hijo	$D_1 : [C_2, C_1], D_2 : [C_3]$	$\{(C_1, C_2)\}$

$$\text{BPD}(\text{Hijo}, \text{Chr2}) = |\{(C_1, C_2)\} \Delta \{(C_1, C_2)\}| = 0$$

Como $\text{BPD} = 0 < \Delta = 1$ (umbral actual), **el hijo es rechazado** por ser demasiado similar a Chr2. $\text{NbNoAdd} += 1$.

Cuando NbNoAdd supere MaxNbNoAdd , el umbral Δ descenderá a 0 y el hijo (costo=18) será aceptado en el siguiente intento, reemplazando al individuo de mayor costo en la población.

Nota Clave

Este ejemplo ilustra un caso curioso: el MA|PM puede *descubrir* la solución óptima (costo 18) en el cruce pero *rechazarla temporalmente* por ser muy similar a un individuo existente. El mecanismo BPD prioriza la diversidad; la solución óptima entra en cuanto baja el umbral. En instancias reales con $n \geq 10$, el espacio de pares es mucho más rico y los rechazos por BPD son menos frecuentes.

Resultado del MA|PM

Tras la incorporación del hijo (cuando $\Delta = 0$), la población contiene un individuo con costo **18**, que coincide con el óptimo global encontrado por la Fuerza Bruta.

Resumen comparativo de los tres algoritmos en esta instancia

Algoritmo	Costo final	Solución óptima	Iteraciones trazadas	Observación
Fuerza Bruta	18	Sí (garantizado)	3 subconjuntos (18 asignaciones)	Exacto, exponencial
ALNS	18	Sí (en este caso)	3 iter. mostradas	Puede atascarse si
MA PM	18	Sí (tras BPD)	1 generación	BPD retrasó pero n

Los tres algoritmos convergen al óptimo en esta instancia pequeña. La diferencia emerge en instancias grandes ($n \geq 50$): la Fuerza Bruta se vuelve intratable, mientras que ALNS y MA|PM encuentran soluciones de alta calidad en tiempo razonable.